
✅ 1️⃣ Overview Section

🎬 Video title

Day 01 – How Does Terraform Work | Intro to IaC (YouTube video in the series) (YouTube)

📌 Main objective of the video

- Introduce Terraform and explain *how infrastructure-as-code (IaC)* works. (DEV Community)
- Provide foundational context needed before writing any Terraform code.
- Set up mental models for declarative provisioning and the Terraform workflow.

📋 Prerequisites assumed

- Basic familiarity with the concept of cloud resources (e.g., AWS EC2, S3).
- Ability to use a terminal/CLI.
- No prior Terraform experience is required (beginner-oriented). (DEV Community)

🛠️ Tools discussed or implied

- **Terraform CLI** (e.g. Terraform v1.x).
- **AWS as a cloud provider** (EC2, IAM etc.).
- HCL (HashiCorp Configuration Language) — Terraform's config syntax.
- AWS API & SDKs (Terraform communicates via these). (HashiCorp Developer)

✅ 2️⃣ Core Concepts Explained

This section breaks down each concept likely covered in an "intro to IaC with Terraform on AWS".

🔷 Infrastructure as Code (IaC)

💡 Definition

A methodology for defining infrastructure in code files instead of manually provisioning via consoles or scripts. (DEV Community)

⚙️ Why it's used

- Consistency and repeatability
- Version control & audit history
- Automation & reduced human error

📍 Real-world relevance

Teams use IaC to provision identical environments (dev/test/prod), prevent "configuration drift," enforce policies, and embed infrastructure in CI/CD. (DEV Community)

🔷 Terraform

💡 Definition

Terraform is a declarative IaC tool by HashiCorp that lets you define infrastructure across cloud providers using HCL. (HashiCorp Developer)

⚙️ Why it's used

- Provider-agnostic: works with AWS, Azure, GCP, etc.
- Declarative: declare a desired end state, not procedural steps.
- State-driven: tracks resources and fixes drifts.

📍 Real-world relevance

Terraform is one of the most widely used IaC tools in DevOps, used for consistent provisioning of VPCs, EC2s, databases, S3, IAM, etc. (HashiCorp Developer)

🔷 Providers

💡 Definition

Plugins that enable Terraform to interact with specific cloud or service APIs. (HashiCorp Developer)

⚙️ Why it's used

Terraform doesn't talk directly to AWS — providers translate config into API calls.

📍 AWS relation

The **AWS provider** lets you create EC2, VPC, IAM, S3, RDS, and many more AWS resources. (HashiCorp Developer)

🔷 Workflow – Write, Plan, Apply, Destroy

This is fundamental to any Terraform project.

1. **Write** — develop .tf config files. (DEV Community)
2. **Init** — initialize and download providers.
3. **Plan** — preview changes (show create/update/delete).
4. **Apply** — execute the changes.
5. **Destroy** — teardown infrastructure. (DEV Community)

✅ 3️⃣ Code Breakdown (Very Important)

Even if the video doesn't show code yet, a standard snippet used in a Terraform intro looks like:

```
# Define the AWS provider

provider "aws" {

  region  = "us-west-2"

}


# Create an EC2 instance
```

```
resource "aws_instance" "example" {
  ami           = "ami-0c55b159cbfafe1f0"
  instance_type = "t2.micro"
}
```

Explanation line-by-line

Line	Component	Purpose
provider "aws"	Provider block	Defines AWS as the target cloud
region = ...	Argument	AWS Region Terraform will work in
resource "aws_instance" "example"	Resource block	Defines an EC2 instance named example
ami	Attribute	AMI ID used to launch EC2
instance_type	Attribute	EC2 instance size

📍 **Dependencies**

Terraform computes implicit dependencies from references (none in this simple snippet).

📍 **Common mistakes**

- Using an incorrect AMI ID (AMI must be valid in the specified region).
- Not running terraform init before plan/apply.

✅ 4️⃣ **Commands Used**

Command	Purpose
terraform init	Initializes the directory, downloads providers
terraform plan	Shows what Terraform *will* do
terraform apply	Creates or modifies infrastructure
terraform destroy	Deletes all managed resources

When to use

- Always start with init.
- Use plan before apply to avoid surprises.
- destroy when cleaning up.

✅ 5 AWS Architecture Explained

Intro tutorials generally include:

User (Terraform CLI)

↓ terraform apply API calls

AWS API → Creates:

EC2 instances

VPC

IAM roles

✅ 6 Best Practices Mentioned

• Use version control (Git).
• Always run plan before apply.
• Store state securely (remote backend).
• Do not commit sensitive values (use variables). (HashiCorp Developer)

✅ 7 Important Interview Questions From This Video

Conceptual

- What is Terraform?
- What is IaC and why use it?

Scenario-based

- You wrote a Terraform config — what happens if you run apply again without changing anything?

Troubleshooting

- Why would Terraform not create an AWS resource even after apply?

✅ 8 Common Errors / Debugging

State drift

Cause: Resources changed outside Terraform.
Solution: Use terraform refresh and import.

Missing provider

Cause: Forget terraform init.
Solution: Run init first.

✅ 9 Summary for Quick Revision

• Terraform lets you *declare* the desired state of infra. (HashiCorp Developer)
• Workflow: Write → Init → Plan → Apply → Destroy. (DEV Community)
• Providers like AWS allow Terraform to reach real clouds.

🔟 Personal Notes Section

- **My Notes:**
  Infrastructure as Code (IaC) = Managing infrastructure using **declarative configuration files** instead of manual console work.
- Terraform is **declarative**, not procedural.
  → We define the *desired state*, Terraform figures out *how* to achieve it.
- Terraform uses **Providers** to communicate with cloud APIs (e.g., AWS provider).
- Terraform maintains a **State file** (terraform.tfstate)
  → Tracks what resources it created
  → Maps real infrastructure to configuration
  → Prevents duplicate creation

Core Workflow:

1. terraform init → initialize & download provider
2. terraform plan → preview execution plan
3. terraform apply → create/update infrastructure
4. terraform destroy → remove infrastructure

- Terraform interacts with AWS through **AWS APIs**, not through the console.
- If nothing changes in configuration:
  → terraform apply results in **No changes**
- Infrastructure becomes:
  - Version-controlled
  - Repeatable
  - Shareable across teams
- Terraform compares:
  - Configuration file
  - State file
  - Real infrastructure
- Manual changes in AWS console can cause **state drift**.

🎯 **For Interview Preparation, Remember:**

- Terraform is **idempotent** (running apply multiple times does not duplicate resources).
- State management is critical in real-world environments.
- Always run plan before apply.
- Terraform is cloud-agnostic (multi-provider support).
